TALLER DE INVESTIGACIÓN APLICADA: ARQUITECTURA DE SOFTWARE SEGURO

Curso: BCD 9220 - Seguridad de Sistemas Digitales

Modalidad: Grupal (4 grupos de 10 integrantes)

Tiempo de ejecución: 1 hora y 20 minutos (80 minutos)

Formato de entrega: Documento PDF subido al campus virtual (cada estudiante debe subir el archivo).

1. Objetivo del Taller

Investigar y aplicar los pilares fundamentales del desarrollo de software seguro en aplicaciones modernas (utilizando React o Angular para el Frontend, y el lenguaje de su preferencia para el Backend). El objetivo es demostrar cómo se implementan controles de seguridad a nivel de código, comprendiendo que la seguridad es un proceso continuo y no un producto.

2. Validación de Carga de Trabajo (Grupos de 10 personas)

Para garantizar que los 10 integrantes del grupo trabajen equitativamente, el equipo debe dividirse en 5 sub-equipos de 2 personas. Cada sub-equipo tendrá un rol específico:

1. Sub-equipo 1 (Investigación Teórica): Define el pilar y las 3 buenas prácticas. Busca referencias en OWASP.
2. Sub-equipo 2 (Arquitectura): Diseña un diagrama de flujo que explique cómo se aplica el pilar desde que el usuario interactúa con el Frontend hasta que el Backend responde.
3. Sub-equipo 3 (Frontend - React/Angular): Desarrolla los ejemplos de código del lado del cliente.
4. Sub-equipo 4 (Backend - Node.js, .NET, Python, etc.): Desarrolla los ejemplos de código del lado del servidor.
5. Sub-equipo 5 (QA y Documentación): Consolida el trabajo, revisa la sintaxis del código, redacta el documento final y prepara al vocero para la exposición.

3. Asignación de Pilares y Requerimientos Técnicos

Cada grupo debe investigar su pilar asignado, definir al menos 3 buenas prácticas y presentar los ejemplos técnicos solicitados. No se aceptarán ejemplos teóricos o pseudocódigo; se requiere código real o configuraciones específicas.

GRUPO 1: Autenticación y Gestión de Sesiones

El reto: Un usuario inicia sesión. ¿Dónde almacena el Frontend la prueba de autenticación sin exponerse a ataques XSS?

- Buenas prácticas a investigar (mínimo 3): Uso de Cookies HttpOnly, implementación de Refresh Tokens, expiración de Access Tokens.
- Referencia Técnica Obligatoria (Deben incluir esto o mejorarlo):
    - *Frontend (React):* Crear un componente de Ruta Protegida (Protected Route) que verifique si el usuario está autenticado antes de renderizar el componente.
    - *Backend (Node.js/Express):* Configurar el servidor para enviar el token en una cookie segura, no en el cuerpo de la respuesta.
      (El grupo debe documentar el código exacto de cómo se configura la cookie con `httpOnly: true, secure: true` *y cómo el frontend verifica la sesión).*

GRUPO 2: Validación de Entradas y Sanitización

El reto: Un atacante intenta inyectar un script malicioso (XSS) o una consulta SQL (SQLi) a través de un formulario.

- Buenas prácticas a investigar (mínimo 3): Validación en el cliente y servidor, sanitización de HTML, uso de consultas parametrizadas.
- Referencia Técnica Obligatoria (Deben incluir esto o mejorarlo):
    - *Frontend (React):* Mostrar el uso de una librería como `DOMPurify` para limpiar el HTML antes de renderizarlo (evitando `dangerouslySetInnerHTML` sin sanitizar).
    - *Backend (Node.js/MySQL):* Mostrar la diferencia en código entre una consulta vulnerable por concatenación (`"SELECT * FROM users WHERE id = " + id`) y una consulta parametrizada segura (`db.query("SELECT * FROM users WHERE id = ?", [id])`).

GRUPO 3: Gestión de Secretos y Protección de Datos

El reto: Proteger las credenciales de la base de datos y las API Keys para que no sean expuestas en el repositorio o en el navegador del cliente.

- Buenas prácticas a investigar (mínimo 3): Uso de variables de entorno, archivo `.gitignore`, cifrado en tránsito (HTTPS/TLS).
- Referencia Técnica Obligatoria (Deben incluir esto o mejorarlo):
    - *Frontend:* Documentar cómo se manejan las variables de entorno públicas en React (`process.env.REACT_APP_...`) y explicar por qué nunca se debe poner una clave privada en el código del cliente.
    - *Backend:* Mostrar un ejemplo de archivo `.env` y cómo se inyecta de forma segura en el código (ej. `process.env.DB_PASSWORD`). Explicar el uso de servicios de gestión de secretos (como AWS Secrets Manager o HashiCorp Vault) para entornos de producción.

GRUPO 4: Control de Acceso y Autorización (RBAC)

El reto: Asegurar que un usuario con rol "Invitado" no pueda ejecutar acciones de "Administrador", incluso si modifica la petición HTTP manualmente.

- Buenas prácticas a investigar (mínimo 3): Principio de mínimo privilegio, implementación de RBAC (Control de Acceso Basado en Roles), validación del lado del servidor para cada endpoint.
- Referencia Técnica Obligatoria (Deben incluir esto o mejorarlo):
    - *Frontend (Angular/React):* Código que oculte un botón de "Eliminar Usuario" si el rol del usuario logueado no es "admin".
    - *Backend (Node.js/Express):* Crear un Middleware de autorización que intercepte la petición y verifique el rol del usuario antes de dejarlo pasar a la función de eliminación.

4. Cronograma de la Clase (80 minutos)

- Minuto 0 - 10 (Briefing): Explicación de la dinámica, asignación de grupos y apertura de salas (Breakout Rooms).
- Minuto 10 - 50 (Investigación y Desarrollo - 40 min): Los sub-equipos trabajan en paralelo. Redacción del documento y desarrollo de los ejemplos técnicos.
- Minuto 50 - 60 (Preparación de Exposición - 10 min): El sub-equipo 5 consolida el PDF y prepara al vocero.
- Minuto 60 - 75 (Exposiciones - 15 min): Cada grupo presenta su pilar y su código técnico (3.5 minutos por grupo).
- Minuto 75 - 80 (Cierre): Conclusiones del docente.

5. Entregable (Subir al Campus Virtual)

Cada estudiante debe subir el documento final en formato PDF. El documento debe contener:

1. Portada: Nombres de los 10 integrantes y la asignación de los 5 sub-equipos.
2. Análisis del Pilar: Definición y las 3 buenas prácticas investigadas.
3. Diagrama de Flujo: Imagen del diseño de la arquitectura de seguridad de su pilar.
4. Evidencia Técnica (Código): El código del Frontend y Backend implementado, con comentarios explicando por qué es seguro.
5. Referencias Bibliográficas: Mínimo 4 fuentes (Documentación oficial, OWASP, etc.).

6. Rúbrica de Evaluación (10 Puntos)

Criterio	Excelente (2.5)	Bueno (1.5)	Insuficiente (0.5)
Profundidad Teórica	Define el pilar con precisión técnica y relaciona las prácticas con OWASP.	Define el pilar, pero las prácticas son genéricas.	Información copiada sin análisis.
Ejemplos Técnicos	Código real, funcional y bien estructurado. Explica claramente la implementación.	Pseudocódigo o fragmentos vagos sin explicación.	No presenta ejemplos técnicos.
Arquitectura	Diagrama claro que conecta el Frontend y el Backend de forma segura.	Diagrama confuso o incompleto.	No incluye diagrama.
Trabajo en Equipo	Se evidencia el trabajo de los 5 sub-equipos. Documento robusto y completo.	Carga de trabajo desigual (2 o 3 personas hicieron todo).	No hay evidencia de roles definidos.